Programming Assignment #5: Boggle

Dinesh Nalliah and Jessica Xu

November 3, 2026

Project Overview/Goals

This programming assignment is about implementing the word game Boggle, where players try to find as many words as they can in a grid of letters. Sixteen cubes with letters on each side are randomly arranged on a 4 x 4 grid. A word counts if it is at least four letters long, if its letters are connected horizontally, vertically, or diagonally, and if no cube is used twice. Longer words are worth more points. After the human players are done, the computer finds every word they missed. The implementation is split up into three parts: the dictionary, the game engine, and the user interface.

The dictionary stores all of the valid words from a given word file. We created the GameDictionary class, which implements the given BoggleDictionary interface. Our goal for GameDictionary was to quickly check whether a string is a word or the start of a word, since the game relies on these checks constantly. We also had the additional specification that lookups needed to be efficient and that the dictionary had to be our own data structure rather than an existing Java library class.

The game engine contains most of the logic for the game. We created the GameManager class, which implements the given BoggleGame interface. Our goal for GameManager was to set up a random board, track each player's words and scores, and check whether guessed words are valid. It also has to find every word on the board for the computer's turn using two different search strategies, one that searches the board and one that searches the dictionary.

Lastly, we must also create a user interface in Boggle.java that lets people actually play the game. Our goal for the UI was to display the board, take in guesses, show players where their valid words are on the board, and display the final scores for the players and the computer.

This assignment also included a peer review process, where we tested other teams' compiled code and used the reviews we received to improve our own solution.

Scope, Quality, and Limitations

The scope of our project includes implementing all of the required Boggle mechanics and both search strategies for the computer's turn. It covers building the dictionary, setting up random boards from the cube file, checking players' guesses, keeping track of scores, and finding all remaining words on the board for the computer. It also includes a text-based user interface that supports multiple players taking timed turns on the same board. Since other teams would be testing our code through the given interfaces, we also focused on keeping the game engine separate from the UI, so that GameManager and GameDictionary could be used on their own.

As for the quality of our implementation, it successfully covers the core mechanics of the game. A guess is only accepted if it is long enough, is in the dictionary, can be traced on the board without reusing a cube, and hasn't already been found by that player. When a word is valid, the letters on its path are shown in uppercase so the player can see where it was found. Both search strategies return the same set of words, and the computer never takes credit for words a player already found. The game is also not limited to the traditional 4 x 4 board, and it can create larger boards by reusing cubes when there are more squares than cubes. We also wrote tests to check the dictionary, word validation, scoring, and both searches, including edge cases like smaller boards and repeated guesses.

Some limitations come from our timer and our text-based interface. The game can't interrupt a player once they start typing. If time runs out while a player is typing a word, the game waits for them to finish and submit it. The late word isn't counted, but the turn still runs past the time limit for as long as the player keeps typing. Another limitation is how we "clear" the screen between turns. Since the UI just prints a large number of blank lines, the next player could scroll up and see the previous player's guesses. Lastly, our dictionary-driven search checks every word in the dictionary against the board without many shortcuts, so it is noticeably slower than the board-driven search. This doesn't affect gameplay at our board size, but it would matter more for larger dictionaries.

Algorithm & Implementation

Trie

The dictionary is stored in a trie, a tree where each edge is one letter and each root-to-node path spells a prefix. Every Node holds an array of 26 child nodes and a boolean isEndOfWord that marks whether the path ending at that node is a complete word. The root represents the empty string. Words that share a beginning, such as "tea", "team" and "teach", share the nodes for "tea", so each common prefix is stored only once.

insert(word) starts at the root and walks the word one character at a time. For each character it computes the index c - 'a'. If that child is null it creates a new Node, then moves into the child. After the last character it sets isEndOfWord = true on the final node. Inserting a word of length L takes O(L) time.

search(word) follows the same walk without creating nodes. It returns false immediately if a character is outside a to z or if the needed child does not exist. If the walk finishes, it returns the final node's isEndOfWord boolean, so a string that is only a prefix (such as "tea" when only "team" was inserted) is correctly rejected.

startsWith(prefix) is identical to search except that it returns true as soon as the walk finishes, without checking isEndOfWord. Any path that exists in the trie is a prefix of at least one word. Both lookups cost O(L) and do not depend on how many words the dictionary holds.

getRoot() returns the root so GameDictionary's iterator can traverse the tree directly.

GameDictionary

GameDictionary implements BoggleDictionary as a thin wrapper around the trie. A boolean isLoaded records whether a dictionary has been read; until it is true, every query returns false and the iterator is empty.

loadDictionary(filename) creates a fresh Trie, then reads the file with a Scanner, one whitespace-separated token at a time. Each token is lowercased and inserted, so lookups are case-insensitive as long as callers also lowercase. After the file is read, isLoaded is set to true. Loading N words with average length L costs O(N·L). If the file does not exist, the FileNotFoundException is caught and a message is printed.

contains(word) returns false if nothing is loaded, otherwise it delegates to Trie.search. It answers in O(L) for a word of length L.

isPrefix(prefix) returns false if nothing is loaded, otherwise it delegates to Trie.startsWith, also in O(L). This is the check the board-driven search uses to stop exploring paths that cannot lead to any word.

iterator() returns a DictIterator over the trie's root, or an empty iterator if nothing is loaded. DictIterator performs a depth-first, pre-order traversal of the trie using an explicit stack instead of recursion, which lets it pause between words. Each stack entry (IterState) stores a node, the string spelled by the path to it, and childIndex, the next child slot to examine.

The iterator always holds the next word in advance in nextWord. Its advance() method works as follows:

1. Look at the top of the stack. If this is the node's first visit (childIndex == 0) and the node ends a word, record its string as nextWord.

2. Scan the node's children from childIndex up to 25. At the first non-null child, push a new state for it with the parent's string plus that letter, and advance childIndex past it.

3. If a child is pushed and a word is waiting, return. If no child remains, pop the node, and return if a word is waiting. Otherwise repeat from step 1.

Because children are visited from a to z and a node's own word is reported before any of its descendants, words come out in alphabetical order ("tea" before "teach" before "team"). The constructor calls advance() once to pre-fetch the first word. hasNext() simply checks whether nextWord is non-null, and next() returns nextWord, calls advance() to find the following word, and throws NoSuchElementException when there is nothing left. The stack never holds more entries than the length of the longest word, so iteration uses very little extra memory.

GameManager

GameManager implements BoggleGame. It stores the board as a lowercase char[][], one HashSet<String> of found words and one score per player, the path of the last added word, the search tactic, and the computer's words and score.

newGame(size, numPlayers, cubeFile, dict) reads every non-blank line of the cube file, trimmed and lowercased, into a list where each line represents all of the faces of a cube. The list is shuffled with Collections.shuffle so that cubes land in random cells, and the board is filled in row-major order: cell (i, j) takes cube i * size + j and shows one of its faces chosen at random. The method then stores the dictionary

and number of players, creates an empty word set and a zero score for each player, sets the last path to null, and selects board-driven search (default).

getBoard() returns the board array in row-major order.

addWord(word, player) validates the guess and, if it is legal, credits the player. It returns 0 without changing anything if the word is null, shorter than 4 letters, not in the dictionary, or already found by that player. Otherwise it lowercases the word and tries each cell of the board as a starting point, calling the recursive helper isValid with a fresh visited array and an empty path. At the first starting cell that succeeds, it computes the score as length - 3 (so a 4-letter word is worth 1 point), adds the word to the player's set and the points to the player's score, saves the path for getLastAddedWord, and stops searching. If no starting cell works, the word is not on the board and 0 is returned.

getLastAddedWord() returns the list of Points saved by the last successful addWord, one per letter in order, each stored as (row, column). It returns null if no word has been added since the game started or the board was reset. A rejected guess does not change it.

setGame(board) replaces the board with the given square array, converting each letter to lowercase. It clears every player's word set and score, sets the last path to null, and resets the result set and the computer's score. The dictionary, player count, and search tactic are kept, as the interface requires.

getAllWords() returns every valid word (4 or more letters, in the dictionary, and formable on the board), using the current tactic. It first resets the result set and the computer score. With SEARCH_BOARD it starts the recursive boardDrivenFind from every cell, sharing one visited array; with SEARCH_DICT it calls dictionaryDrivenFind. Both helpers also credit the computer with every word that no player found.

setSearchTactic(tactic) maps the enum onto a boolean: SEARCH_BOARD sets board-driven search and SEARCH_DICT sets dictionary-driven search. A null tactic is replaced with SEARCH_DEFAULT, which is SEARCH_BOARD.

getScores() returns the array of player scores, indexed by player number.

Helper methods

isValid(i, j, s, visited, path) is a recursive backtracking search that checks whether the string s can be spelled starting at cell (i, j). It fails if the cell is off the board, already used in the current path, or does not match the first letter of s. If s has one letter left and it matches, the cell is added to the path and the search succeeds. Otherwise it marks the cell visited, adds it to the path, and tries each of the 8 neighbors (using the ROW_OFFSETS and COL_OFFSETS arrays) on the rest of the string. If any neighbor succeeds it returns true immediately; if all fail it removes the cell from the path and unmarks it, so other paths may reuse that cell. addWord and dictionaryDrivenFind both rely on it.

boardDrivenFind(i, j, words, word, visited) enumerates paths on the board. It returns if the cell is off the board or already in the current path. Otherwise it appends the cell's letter to the current string. If the string is a dictionary word of 4 or more letters, it is added to the result set and, unless a player already found it, to the computer's words and score. If the string is not a prefix of any dictionary word, no longer path through it can succeed, so the method returns. Otherwise it marks the cell visited, recurses into all 8 neighbors, and unmarks the cell when done. Because Java strings are immutable, each call extends its own copy of the string and nothing needs to be undone.

dictionaryDrivenFind() iterates over every dictionary word of 4 or more letters. A word a player already found is added to the result set directly, since addWord already verified it on this board. Every other word is tested with isValid from each starting cell; at the first success it is added to the result set and credited to the computer, and the remaining cells are skipped.

foundByPlayer(word) returns true if any player's word set contains the word. Both search helpers use it to keep the computer from earning points for player words.

The remaining public methods are simple accessors used by the console interface: getComputerScore(), getComputerWords(), getWords(player), and resetPath(), which clears the highlighted path between turns.

Discussion of Efficiency

Why a trie?

A trie was chosen because it supports checking the existence of a word and if a prefix of a word exists in time proportional to the length of the string, regardless of dictionary size. The prefix question is what separates tries from other data structures, as the common alternatives handle it poorly.

What's the order of the algorithms?

Data Structure	Search Complexity	Prefix-Search Complexity
HashSet	O(L)	O(N * L)
TreeSet	O(L log N)	O(L log N) *assuming finding the prefix range*
Trie	O(L)	O(L)

Here N is the number of words and L is the length of the string being checked. The trie also shares common prefixes, so related words like "play", "player" and "playing" reuse the same nodes, and its depth-first order produces words alphabetically for free, which the iterator relies on. The major drawback is memory, as each node reserves 26 child references even though most are null, so a large word list creates many mostly empty arrays.

A small inefficiency in our algorithm can be seen in boardDrivenFind, where it calls contains and isPrefix separately, and each walks the trie from the root, so a path of length L costs O(L) per step. If we could pass the current trie node down the recursion it would make each step O(1).

Is Board-driven or dictionary-driven search better?

The two tactics do the same job from opposite directions, and their costs depend on different things.

Board-driven search explores paths on the board and stops a path as soon as it is not a dictionary prefix. Without optimizing, a 4x4 board has millions of simple paths, but almost all of them die within a few letters because no English word starts that way. Its cost is therefore driven by the board: the number of letter sequences on it that are also real prefixes. Adding more words to the dictionary adds some new live prefixes, but that number grows slowly and is capped by what the board can spell.

Dictionary-driven search checks every word in the dictionary against every starting cell. Each individual check is cheap, since most fail on the first or second letter, but the total work grows linearly with the dictionary: roughly N words times n^2 starting cells for an $n \times n$ board, with nearly all of that effort spent on words that are not on the board.

Small dictionaries favor dictionary-driven search. With only a few hundred or few thousand words, checking each one is fast and predictable, and board-driven search gains little from its optimizations while still starting from every cell and making two trie lookups per step.

Large dictionaries favor board-driven search. With a full word list of hundreds of thousands of words, dictionary-driven search spends many calls rejecting words that cannot appear, while board-driven search does about the same amount of work as before because it only follows paths the board can actually form. The board size does the opposite, as a larger board multiplies the paths board-driven search must explore, while dictionary-driven search grows only with the number of starting cells, so very large boards narrow the gap.

Game Design

For our user interface, we chose a text-based console interface instead of a graphical window. The project specifications allowed either option, and we decided that a console interface was the simpler choice. Neither of us has much experience building graphical user interfaces in Java, so learning a library like Swing would have taken time away from the main parts of the assignment, like the dictionary and the search algorithms. A console interface also kept Boggle.java short and easy to follow. It only has to print the board and read in guesses, while all of the actual game logic stays in GameManager.

When the game starts, it asks how many players are playing. If the input is blank or isn't a valid number, it defaults to one player. Then each player takes a timed turn on the same board. During a turn, the board is printed as a 4 x 4 grid of lowercase letters, and the player is prompted to type a guess. The prompt also shows how much time is left and reminds the player that they can type "q" to end their turn early. We picked "q" because it is shorter than four letters, so it could never be mistaken for a real guess.

After each guess, the game prints a short message saying how many points were added, that the word was already guessed, or that the word was invalid. If the word was valid, the board is printed again with the letters of that word in uppercase. The specifications asked us to visually show where a word was found, and since we couldn't use colors in a basic console, changing the case of the letters was the clearest option. It lets the player see the whole path at once, which we found much easier to read than a list of coordinates.

Since every player shares the same screen, we had to make sure the next player couldn't see the previous player's words. Between turns, the game prints blank lines to push the old output off the screen and then waits for the next player to press ENTER before starting their turn. This gives players time to switch seats before the timer starts.

Once every player has finished, the game shows each player's score and the words they found. It then shows the words that the computer found that no player had guessed, along with the computer's score. Finally, it asks whether the players want to play again. If they do, a new random board is set up, and the dictionary is reused so it doesn't have to be loaded again.

```
Player 1: 11
Words: [lins, vise, jives, veil, luny, lies, rise, yule, soon, jive]



Words on board: 76
Words not found players (Computer Words): [serin, nisei, vino, revs, rins, siver, vinier, jin
n, ionise, vinos, vins, rive, snivel, linos, vies, vier, virl, foin, info, noisier, levins, o
sier, yules, levin, vein, lives, liver, ires, reis, riel, rein, reins, revision, vision, resi
n, viler, evil, sire, vires, lesion, lire, nosier, rile, elision, lino, linn, leis, foins, so
nny, nooser, lier, noise, ions, vile, live, rinse, nose, nisi, noose, veins, ivies, riles, ji
ns, rives, fons, infos]
Computer score: 116
Play again? (y/n): n
```

Figure 1: Sample game end screen

Testing

We used both black box and white box testing to evaluate our implementation. Black box testing checked whether each method returned the expected results for different inputs, while white box testing focused on parts of our code that we knew could cause problems, like the trie traversal and the recursive searches. Most of our JUnit tests used a fixed board set through setGame(), since a random board would make it impossible to know which words should be found. We also ran the full game ourselves to test the user interface and to catch problems that our individual tests missed.

GameDictionary

Our GameDictionary tests focused on contains(), isPrefix(), and the iterator. For contains(), we checked a normal word, the first word in words.txt ("aa"), and the last word ("zymurgy"). The last word was important because the file doesn't end with a newline, so we wanted to make sure it was still read in correctly. We also checked that a prefix of a word like "aardvar" was not counted as a word, and that the empty string was not a word.

For isPrefix(), we checked that a single letter, a partial word, and a full word all counted as prefixes, since a full word is a prefix of itself. We also tested a string that goes one letter past the end of the last word, and a string like "xq" that no word starts with.

The iterator tests were more white box, since we knew our iterator walks through the trie using a stack. We iterated through the entire dictionary and checked that every word came out in alphabetical order, that every word returned was actually in the dictionary, and that the total count matched the 113,764 words in words.txt. We also tested edge cases: calling hasNext() multiple times without skipping a word, running two iterators at the same time without them affecting each other, and calling next() after the last word, which should throw a NoSuchElementException.

The following table summarizes the main tests we performed for GameDictionary:

Test	Input Specifications	Expected Behavior
Contains	Normal word, first word, last word	Returns true
Contains (invalid)	Prefix only, non-word, empty string	Returns false
Prefixes	Single letter, partial word, full word	Returns true
Prefixes (invalid)	Past the last word, “xq”	Returns false
Iterator order	Iterate through whole dictionary	All 113,764 words in alphabetical order
Iterator edge cases	Repeated hasNext(), two iterators, next() at end	Correct words, then NoSuchElementException

GameManager

For GameManager, we used a fixed 4 x 4 test board and a two-player game. We first tested addWord() with valid words of different lengths to check that four-letter words were worth 1 point and five-letter words were worth 2 points, and that each player’s score was tracked separately. We then tested every way a guess could be invalid: an empty string, a word that is too short (“pie”), a word that isn’t on the board (“zebra”), a string on the board that isn’t a word (“kitw”), and a word that would need to reuse the same cube (“twit”). None of these should give points or change the score. We also tested duplicates: the same player guessing the same word twice should get no points the second time, but a different player should still be able to get points for it.

For getLastAddedWord(), we checked that it returns null before any word has been added, as the specifications require. After adding “kite,” we checked that it returned the exact coordinates of each letter in order, and that guessing an invalid word afterwards did not change the saved path.

To test getAllWords(), we found the list of every valid word on our test board and checked that both search tactics returned exactly that set of 24 words. We also switched to a 2 x 2 board to make sure the results only reflected the new board and that our game worked on a board size other than 4 x 4. Lastly, we tested that setGame() reset all scores and word lists, so a player could add the same word again after the reset.

The following table summarizes the main tests we performed for GameManager:

Test	Input Specifications	Expected Behavior

Valid words	4 and 5 letter words, two players	Correct points, scores tracked per player
Invalid words	Empty, too short, not on board, not a word, reused cube	0 points, score unchanged
Duplicates	Same word twice, then by other player	0 points the second time, other player still scores
Last added word	Before any word, after “kite,” after invalid word	null, then correct path, then path unchanged
Both search tactics	Board search and dictionary search on test board	Both return the same 24 words
Smaller board	2 x 2 board	Only “note” and “tone” found
Resetting the game	Call setGame() after adding words	Scores and word lists reset

This assignment taught us that tests are only as good as what they actually check. Our getAllWords() test passed the whole time, even though the computer’s score was wrong, because it only compared the words found and not the points. Comparing the two search tactics against each other turned out to be really useful, since they should always give the same results, so any difference between them points to a bug. Like in our Tetris project, actually playing the game also caught problems that our individual tests missed.

Interesting Results

One interesting result we found was that because both recursive searches backtrack, we don’t need to create a new visited array before each call. In getAllWords(), a single array is shared across all sixteen starting cells: every call that marks a cell visited unmarks it before returning, so by the time the search comes back to the top level, the array is all false again. This observation helped improve our efficiency, as we didn’t have to allocate a new array every iteration.

Another surprise came from where we placed an early return in boardDrivenFind. Our first attempt at keeping player-found words away from the computer checked each player’s word set and returned immediately on a match. The word was correctly excluded, but the return also skipped the prefix check and the recursion, so the search stopped extending that path. If a player found “test”, the computer could never find words that began with “test”. This caused us to make two conditions, as whether to credit a word is independent of whether to keep searching past it. The method now records the word only if no player found it, then always falls through to the prefix check.

Assumptions

GameDictionary

We assume that the word file has words separated by whitespace and that every valid word only uses the letters a through z. Our trie only has 26 children per node, so any word with other characters, like an apostrophe or a hyphen, is skipped when loading instead of being added. Words are converted to lowercase as they are read in, so the dictionary is not case-sensitive. The specifications say the file will be in alphabetical order, but our trie doesn't rely on this, and the iterator returns the words in order either way. We assume that loadDictionary() is called before the dictionary is used. If it isn't, contains() and isPrefix() return false and the iterator has no words. We also assume that the dictionary is not reloaded while an iterator is being used, since that iterator would keep going through the old words. Lastly, we treat the empty string as a prefix of every word but not as a word itself.

GameManager

We assume that every board passed to setGame() is square, since our code uses the number of rows as the size in both directions. We also assume that each square holds a single letter. Real Boggle has a "Qu" cube, but our game treats Q as its own letter, so words like "quit" need a separate Q and U next to each other. For newGame(), we assume that the size is positive, that there is at least one player, and that each non-blank line of the cube file is one cube. If the board has more squares than there are cubes, we assume it is okay to reuse cubes.

For addWord(), we assume that players are numbered from 0 to numPlayers - 1, and a guess from any other player number is worth 0 points. Guesses are not case-sensitive. We assume that getAllWords() is called after all players are done guessing, since the computer only skips words that players had already found at the time it is called. getComputerScore() and getComputerWords() only have meaningful values after getAllWords() has been called. We also assume that the code using GameManager doesn't modify the arrays and lists returned by getBoard(), getScores(), and getLastAddedWord(), since these return our actual internal data instead of copies.

Boggle UI

We assume that words.txt and cubes.txt are in the folder the game is run from. Since all players share the same screen, we assume that players don't scroll up to see the previous player's guesses, and that each player presses ENTER to submit a guess, because the game can't cut a player off while they are typing. The UI always uses a 4 x 4 board, even though GameManager supports other sizes. Players are numbered starting from 1 in the UI, while GameManager numbers them starting from 0.

Problems Encountered

One problem we encountered while writing our recursive searches was figuring out how to backtrack correctly. Both searches use a visited array to make sure the same cube isn't used twice in one word. In our first version of the board-driven search, we marked a square as visited when the search reached it, but we never set it back to false after the search was done exploring from that square. This meant that once a square was used in one path, it stayed marked for the rest of the search. Our search ended up missing words that started the same way as a word it had already explored, since the squares they needed were still marked as used. We fixed this by setting the square back to false after checking all eight of its neighbors, so the square becomes available again once the search backs out of it. We used the same idea in our addWord() search, which also removes the last point from the saved path when a path turns out to be a dead end. This taught us that backtracking means undoing everything a step changed, not just returning from it.

Another problem was implementing the timer for each player's turn. We tried a few different approaches, including using Java's Timer class to end the turn after a set amount of time. The issue was that the game waits for the player to type a guess and press ENTER, and while it is waiting, nothing else in that part of the program can run. A Timer would run on a separate thread, so stopping the player in the middle of typing would have required a much more complicated setup than we wanted for a text-based game. We ended up with a simpler approach: we record the time when a turn should end, and after each guess we check whether that time has passed. If it has, the turn ends and the late word isn't counted. This works well, but it is also why the game can't cut a player off while they are typing, as mentioned in our limitations.

We also used AI to help us review our code for bugs, which helped us catch several small issues we had missed. The most significant one was that the board-driven search gave the computer extra points. If a word could be traced on the board in two different ways, our search found it twice. The word was only stored once, since we kept the words in a set, but its points were added each time it was found. We didn't notice at first because our tests only checked which words were found, not the computer's score. We fixed this by only adding points when a word is added to the set for the first time.

The lesson we took from these bugs was that code can seem to work during normal gameplay and still be wrong in ways that only show up under different conditions. In this case, those conditions were a word with two possible paths and a board size we never played on.

Social Impact

Boggle relies on a dictionary to decide which words are valid. In our game, if a word isn't in words.txt, the player is told it is invalid, no matter how many people actually use it. Many words that Gen Z uses every day come from African American English (AAE), but they were looked down on when Black Americans first used them. African American Vernacular English (AAVE) has its own grammar and pronunciation rules, but many Americans see it "as slang or as being inferior to standard English" (Research Starters 2022). Linguists disagree, and the article notes that "the general consensus among

linguists is that these views are flawed" (Research Starters 2022). Still, studies show that people who use AAVE are more likely to face negative outcomes in school and at work.

Even though AAVE has been stigmatized, many of its words are now a normal part of how young people talk. Linguistics experts have said that many of Gen Z's most popular slang words can be traced back to African American Language (Francis 2026). For example, "rizz," Oxford University Press's 2023 word of the year, was popularized by Black streamer Kai Cenat, while "slay" and "periodt" come from the Black queer community and drag culture. Many of these terms started in Black subcultures like early hip-hop and underground drag, where they weren't respected by the mainstream. One linguist pointed out that the same language that is considered uneducated when Black people use it is often celebrated when non-Black people use it. Critics have also noted that the Black origins of these words are often erased.

Our own dictionary shows this gap. "Slay" is in words.txt and would be accepted in our game, but words like "finna," "bussin," "periodt," "lowkey," and "fleek" are not, even though they are long enough to count and widely used. A player who speaks AAE could find a real word on the board and still be told it is invalid. This might seem small for a word game, but word lists like ours are also used for spell checkers and content moderation. When a list only includes words once they become mainstream, it ends up deciding whose language counts as "real," so developers should think about where their word lists come from and who might be left out.

Karma

To find the total number of possible board configurations, we started with the number of ways the cubes can be arranged on the board. There are 16 cubes, so there are 16! ways to place them in the 16 squares. Each cube can also land on any of its sides, but some cubes have repeated letters, so we only counted the distinct letters on each cube. In cubes.txt, six cubes have 6 distinct letters, seven cubes have 5, and three cubes have 4, giving 6^6 × 5^7 × 4^3 possible combinations of letters facing up. Lastly, we divided by 8 to account for symmetry. A board that is rotated or flipped is technically the same board as before, since it contains the exact same words, and every board can be rotated four ways and flipped. This gives us (16! × 6^6 × 5^7 × 4^3) / 8, or about 6.1 × 10^23 possible boards using the given cubes.
To find the board configuration with the highest score empirically, we could call getAllWords() on every possible board and keep the one where the computer gets the highest score. However, with about 6.1 × 10^23 possible boards, this wouldn't be realistic to actually run.

Theoretically, we can reason about what a high-scoring board would look like. Since longer words are worth more points, a good board should make it easy to form long words. One way to do this is to place common word endings, like "-ing," "-ed," and "-er," near each other on the board so that many words can share them. The board should also be made up of the most common letters in the alphabet and skip rare letters like X and Q, which appear in very few words. Vowels should also be spread apart across the board so that most letters have a vowel next to them, which makes more paths into real words.

Time Log

The log below shows the number of hours split between us for the assignment:

- 10/21
    - Dinesh: Driving for 1 hr
    - Jessica: Driving for 1 hr
- 10/22
    - Dinesh: Driving for 2 hrs
- 10/26
    - Dinesh: Driving for 2 hrs
    - Jessica: Driving for 1 hr
- 10/27
    - Dinesh: Driving for 1 hr
    - Jessica: Driving for 2 hrs
- 10/28
    - Dinesh: Driving for 1 hr
    - Jessica: Driving for 1 hr
- 11/3
    - Dinesh: Driving for 1 hr, worked on report for 1 hr
    - Jessica: Working solo for 1 hr, worked on report for 1.5 hrs

Pair Programming

Boggle was another multi-week project, and working with a partner made every part of it more manageable, from planning our data structures to finding bugs.

One of the biggest benefits of having a partner was being able to share ideas and fill in each other's knowledge gaps. When we first started planning the dictionary, we had to figure out how to store over 100,000 words in a way that made checking for words and prefixes fast. We had heard of the Trie data structure before, so after talking through how it would work, we decided it was a good fit for both contains() and isPrefix(). We had already started implementing our Trie before Thai, our TA, brought it up in the discussion section. If either of us had been working alone, we might have spent a lot more time on a slower design before learning about Tries.

Pair programming also made it easier to catch mistakes and debug our code. When one person was driving, the other stayed engaged and followed along with their code implementation. This allowed the one overseeing to catch small bugs, like typos or wrong indices, as they were being written. These kinds of bugs are easy to fix in the moment, but they can be a pain to track down later once they are buried in the rest of the code. Having a second person checking each line as it was written meant we spent less time debugging later on.

Lastly, having a partner helped us understand the recursion behind our searches. Recursive backtracking can be hard to follow just by reading the code, so we often talked through what the search would do step by step on a small example board. Explaining the logic out loud to each other made it easier to notice when something didn't make sense, which is how we figured out problems like not resetting the visited array. Having to explain the logic also made sure both of us actually understood it, instead of only the person who wrote it.

Overall, pair programming made Boggle much easier to work through. It let us combine what we each knew, catch mistakes earlier, and make sure we both understood how our code worked.

Works Cited

"African American Vernacular English." *Research Starters*, EBSCO, 2022, https://www.ebsco.com/research-starters/social-sciences-and-humanities/african-american-vernacular-english/. Accessed 3 Nov. 2026.

Humiston, Moriah, et al. "Appreciation or Appropriation? How Black Culture Is Shaping Gen Z Slang." *NBC 7 San Diego*, NBC San Diego, 3 Apr. 2026, https://www.nbcnews.com/news/us-news/appreciation-appropriation-black-culture-shaping-gen-z-slang-rcna265993. Accessed 3 Nov. 2026.

"Trie Data Structure in Java." GeeksforGeeks, GeeksforGeeks, 27 May 2024, www.geeksforgeeks.org/java/trie-data-structure-in-java/.